FACULTÉ DES SCIENCES ET DES TECHNOLOGIES(FST)

TD N°1_ Modélisation mathématique

COURS: Modélisation mathématique

PRÉNOM: Lens Sandro

NOM: PÉTIOTE

NIVEAU: L3

DATE: 28/03/2026

Objectif de ce TD

Comprendre les ensembles de caractères et l'entropie.
Comprendre un chiffrement symétrique simple et la rotation des caractères.
Comprendre le chiffrement par clé avec opérations bit à bit.
Relier entropie, longueur de clé et alphabet.

TD 1 : Génération de clés aléatoires

Code

```
import string

import secrets

import math


# Fonction pour calculer l'entropie d'une clé

def entropy(key, charset_size):

    # Formule : entropie = longueur * log2(taille_charset)

    return len(key) * math.log2(charset_size)


# Différents ensembles de caractères

charsets = {

    "ascii_lowercase": string.ascii_lowercase,

    "ascii_letters": string.ascii_letters,

    "digits": string.digits,

    "ascii_letters + digits + punctuation": string.ascii_letters + string.digits +
string.punctuation

}


# Longueurs de clés à générer

lengths = [8, 16, 32]
```

```python
# Génération et calcul
for name, charset in charsets.items():
    print(f"\nEnsemble : {name}")
    for length in lengths:
        key = ''.join(secrets.choice(charset) for _ in range(length))
        H = entropy(key, len(charset))
        print(f"Clé ({length} caractères) : {key}")
        print(f"Entropie ≈ {H:.2f} bits")
```

Test

pa.java | ffffff.md 9+, U | modelisation.py 1, U | Sa Untitled-2 | de Untitled-1

C: > Users > Lens Sandro Petiote > Desktop > projet python > modelisation.py > ...

```python
import string
import secrets
import math

# Fonction pour calculer l'entropie d'une clé
def entropy(key, charset_size):
    # Formule : entropie = longueur * log2(taille_charset)
    return len(key) * math.log2(charset_size)

# Différents ensembles de caractères
charsets = {
    "ascii_lowercase": string.ascii_lowercase,
    "ascii_letters": string.ascii_letters,
    "digits": string.digits,
    "ascii_letters + digits + punctuation": string.ascii_letters + string.digits + string.punctuation
}

# Longueurs de clés à générer
lengths = [8, 16, 32]

# Génération et calcul
for name, charset in charsets.items():
    print(f"\nEnsemble : {name}")
    for length in lengths:
        key = ''.join(secrets.choice(charset) for _ in range(length))
        H = entropy(key, len(charset))
        print(f"Clé ({length} caractères) : {key}")
        print(f"Entropie ≈ {H:.2f} bits")
```

```python
import math

# Fonction pour calculer l'entropie d'une clé
def entropy(key, charset_size):
    # Formule : entropie = longueur * log2(taille_charset)
    return len(key) * math.log2(charset_size)
```

PROBLEMS 15 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Entropie ≈ 45.60 bits
Clé (16 caractères) : tvcbyHJVdbIvnLvb
Entropie ≈ 91.21 bits
Clé (32 caractères) : tbAycvlIsrsAnWsxikGWsPAJQJXYAuCo
Entropie ≈ 182.41 bits

Ensemble : digits
Clé (8 caractères) : 94245712
Entropie ≈ 26.58 bits
Clé (16 caractères) : 1182503867701301
Entropie ≈ 53.15 bits
Entropie ≈ 53.15 bits
Clé (32 caractères) : 26680962705183615306071261926652
Entropie ≈ 106.30 bits

Ensemble : ascii_letters + digits + punctuation
Clé (8 caractères) : `<w>`#_~
Entropie ≈ 52.44 bits
Clé (16 caractères) : Bi=eeCf9u$&5O|Ak
Entropie ≈ 104.87 bits
Clé (32 caractères) : m$Si7=:"!4\\H9"T2GcG!cas>OYAShPV
Entropie ≈ 209.75 bits
PS C:\Users\Lens Sandro Petiote>
```

TD 2 : Chif rement César

Code

```python
import string


def caesar_cipher(message, shift):
    result = ""
    for char in message:
        if char.islower():
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
```

```
        elif char.isupper():
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.isdigit():
            result += chr((ord(char) - ord('0') + shift) % 10 + ord('0'))
        else:
            result += char
    return result

def caesar_decipher(cipher, shift):
    return caesar_cipher(cipher, -shift)

# Exemple
msg = "Hello123"
cipher = caesar_cipher(msg, 3)
print("Chiffré :", cipher)
print("Déchiffré :", caesar_decipher(cipher, 3))
```

Test

pa.java | fffff.md 9+, U | modelisation.py 2, U | Sa Untitled-2 | de Untitled-1

C: > Users > Lens Sandro Petiote > Desktop > projet python > modelisation.py > ...

```python
import string

def caesar_cipher(message, shift):
    result = ""
    for char in message:
        if char.islower():
            result += chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
        elif char.isupper():
            result += chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
        elif char.isdigit():
            result += chr((ord(char) - ord('0') + shift) % 10 + ord('0'))
        else:
            result += char
    return result

def caesar_decipher(cipher, shift):
    return caesar_cipher(cipher, -shift)

# Exemple
msg = "Hello123"
cipher = caesar_cipher(msg, 3)
print("Chiffré :", cipher)
print("Déchiffré :", caesar_decipher(cipher, 3))
```

PROBLEMS 16 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
Entropie ≈ 209.75 bits
PS C:\Users\Lens Sandro Petiote> & "C:/Users/Lens Sandro Petiote/AppData/Local/Programs/Python/Python314/python.exe
 Petiote/Desktop/projet python/modelisation.py"
Chiffré : Khoor456
Déchiffré : Hello123
PS C:\Users\Lens Sandro Petiote>
```

TD 3 Chif rement XOR simple

Code

```
import string

import secrets


def xor_cipher(message, key):

    return ''.join(chr(ord(m) ^ ord(k)) for m, k in zip(message, key))


# Générer une clé aléatoire de même longueur
```

```
message = "BonjourCrypto"
charset = string.printable
key = ''.join(secrets.choice(charset) for _ in range(len(message)))

cipher = xor_cipher(message, key)
decrypted = xor_cipher(cipher, key)

print("Message original :", message)
print("Clé :", key)
print("Chiffré :", cipher)
print("Déchiffré :", decrypted)

# Entropie de la clé
import math
entropy = len(key) * math.log2(len(charset))
print(f"Entropie ≈ {entropy:.2f} bits")
```

Test

pa.java | fffff.md 9+, U | modelisation.py 2, U | Sa Untitled-2 | de Untitled-1

C: > Users > Lens Sandro Petiote > Desktop > projet python > modelisation.py > ...

```python
import string
import secrets

def xor_cipher(message, key):
    return ''.join(chr(ord(m) ^ ord(k)) for m, k in zip(message, key))

# Générer une clé aléatoire de même longueur
message = "BonjourCrypto"
charset = string.printable
key = ''.join(secrets.choice(charset) for _ in range(len(message)))

cipher = xor_cipher(message, key)
decrypted = xor_cipher(cipher, key)

print("Message original :", message)
print("Clé :", key)
print("Chiffré :", cipher)
print("Déchiffré :", decrypted)

# Entropie de la clé
import math
entropy = len(key) * math.log2(len(charset))
print(f"Entropie ≈ {entropy:.2f} bits")
```

PROBLEMS 20 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
Chiffré : Khoor456
Déchiffré : Hello123
PS C:\Users\Lens Sandro Petiote> & "C:/Users/Lens Sandro Petiote/AppData/Local/Programs/Python/Python314/python.exe" "
 Petiote/Desktop/projet python/modelisation.py"
Message original : BonjourCrypto
Clé : m 9tTekW]:fJ|
Chiffré : /fW;/C>
Déchiffré : BonjourCrypto
Entropie ≈ 86.37 bits
PS C:\Users\Lens Sandro Petiote>
```

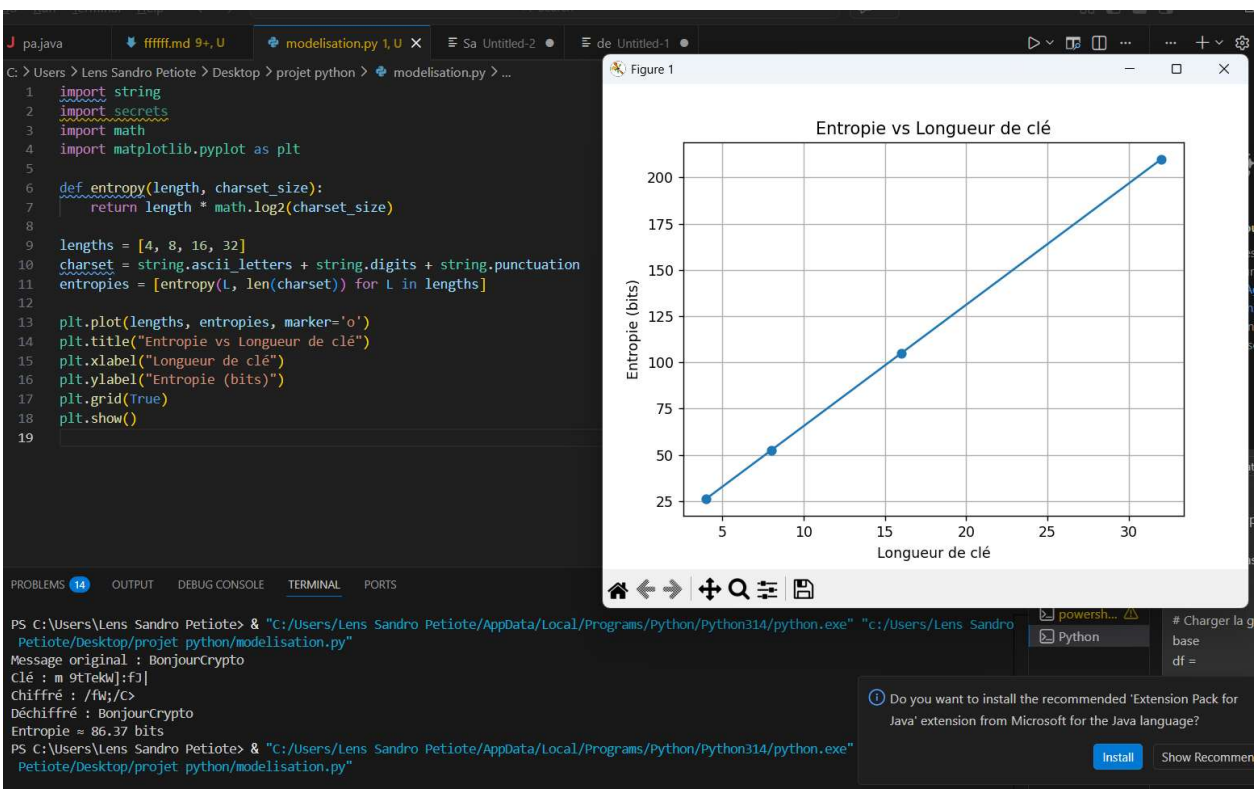
TD 4 : Génération et visualisation de l'entropie

Code

```
import string

import secrets

import math

import matplotlib.pyplot as plt


def entropy(length, charset_size):

    return length * math.log2(charset_size)
```

```
lengths = [4, 8, 16, 32]
charset = string.ascii_letters + string.digits + string.punctuation
entropies = [entropy(L, len(charset)) for L in lengths]

plt.plot(lengths, entropies, marker='o')
plt.title("Entropie vs Longueur de clé")
plt.xlabel("Longueur de clé")
plt.ylabel("Entropie (bits)")
plt.grid(True)
plt.show()
```

Test



Coclusion.

Ce TD m'a permis de comprendre les ensembles de caractères et l'entropie, un chiffrement symétrique simple et la rotation des caractères, chiffrement par clé avec opérations bit à bit et savoir relier entropie, longueur de clé et alphabet, donc la tache a été réussie.